Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 28 de octubre de 2016

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 65 + ULTIMO_DIGITO_PADRON;
  G = 'C';
  A = &H;
  D = &G;
  B = &A;
  cout << *A <<  *D << **B << endl;

  C = new int[3];
  F = C + 2;
  C[1] = H;
  (*A) = 70;
  (*C) = H;
  (*F) = 68;
  C[2]++;
  cout << C[2] << C[1] << C[0] << endl;

  (**B) = C[0] + 1;
  A = C + 2;
  (**B) = H;
  C[0] = *A;
  (*F) = A[0] - 3;
  cout << C[2] << C[1] << C[0] << endl;

  E = (char*) A;
  C[1] = (**B) - 4;
  cout << (*D) << (*E) << G << endl;

  // liberar la memoria
  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura simplemente enlazada el siguiente método, indicando pre y post:

```
T remover(unsigned int posicionElemento);
```

3. Implementar el método `'generarInsumosFaltantes'` de la clase `'IngenieroQuimico'` a partir de las siguientes especificaciones:

```
class IngenieroQuimico {

  public:

    /* post: crea y devuelve una nueva Lista con Compuestos faltantes para completar la prepación de las Soluciones dadas
     *       en la Lista 'solucionesAPreparar' a partir de los Compuestos dados en 'insumosExistentes'
     */
    Lista<Compuesto*>* generarInsumosFaltantes(Lista<Solucion*>* solucionesAPreparar,
                                               Lista<Compuesto*>* insumosExistentes)
};
```

```
class Solucion {

  public:

   /* post: crea la solución con el código especificado, sin
    *       compuestos asociados
    */
    Solucion(string codigo);

    string obtenerCodigo();

    /* post: devuelve los Compuestos requeridos para preparar
     *       la Solución
     */
    Lista<Compuesto*>* obtenerCompuestos()

};

enum Unidad {

  KILO; LITRO;
};
```

```
class Compuesto {

  public:

    /* post: crea el Compuesto, identificado por 'nombre',
     *       con cantidad 0 de la Unidad 'unidadDeMedida'
     */
    Compuesto(string nombre, Unidad unidadDeMedida);

    /* post: devuelve el nombre que identifica el Compuesto
     */
    string obtenerNombre();

    /* post: devuelve la Unidad en la que se mide la cantidad.
     */
    Unidad obtenerUnidad();

    float obtenerCantidad();

    void cambiarCantidad(float cantidad);
};
```

4. Diseñar la especificación e implementar el TDA **Turnador**. Un Turnador distribuye y controla el flujo de atención de turnos. Debe proveer operaciones para:

- Crear el Turnador con la cantidad máxima de turnos en espera.
- Devolver el turno actual.
- Llamar al siguiente turno.
- Entregar un turno.
- Devolver la cantidad actual de turnos de espera.
- Devolver la cantidad total de turnos atendidos.
- Devolver el máximo histórico de turnos en espera.



Nota: el turno siempre debe ser un valor comprendido entre 1 y la máxima cantidad de turnos en espera. Cuando se llega al máximo, debe continuar con 1.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

Solución 1:

```
***0***
65C65
696570
686571
CDC
***1***
66C66
696670
686671
CDC
***2***
67C67
696770
686771
CDC
***3***
68C68
696870
686871
CDC
***4***
69C69
696970
686971
CDC
***5***
70C70
697070
687071
CDC
***6***
71C71
697170
687171
CDC
***7***
72C72
697270
687271
CDC
***8***
73C73
697370
687371
CDC
***9***
74C74
697470
687471
CDC
```



Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas